Craig Gotsman

NJIT New Jersey Institute of Technology

Chapter 10

Indexing Databases

1

Popular SQL Queries

Equality Query (point lookup): Search for a single specific value.

```
SELECT * FROM EMPLOYEE WHERE Ssn = '987987987';
```

Range Query: Look for data within a set of boundary values.

```
SELECT Fname, Lname FROM EMPLOYEE WHERE Salary BETWEEN 25000 AND 35000;
```

Join Query: Link two tables together, typically using a foreign key.

```
SELECT Lname FROM EMPLOYEE E, DEPARTMENT D WHERE E.Dno == D.Dnumber AND D.Dname
== 'Research';
```

Multi-Condition Query: Filter by more than one column at once.

```
SELECT * FROM EMPLOYEE WHERE Sex = 'M' AND Dno == 4;
```

Aggregation Query: Operate on multiple rows together.

```
SELECT AVG(Salary) FROM EMPLOYEE GROUP BY Sex;
```

2

```
SELECT * FROM EMPLOYEE
WHERE Ssn = '987987987';
```

If n employees, takes $O(n)$ time.

```
SELECT Lname FROM
EMPLOYEE E, DEPARTMENT D
WHERE E.Dno == D.Dnumber
  AND D.Dname == 'Research';
```

If n employees and m departments, takes $O(nm)$ time. If Dname a candidate key, then $O(n+m)$.

```
SELECT Lname,Pname FROM
EMPLOYEE E, WORKS_ON WO, PROJECT P
WHERE E.Ssn = WO.Essn
  AND WO.Pno = P.Pnumber;
```

If n employees, m project assignments and k projects, takes $O(nmk)$ time.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

3

Locating Rows (Lookup)

EMPLOYEE

FName	LName	…
Abraham	Smith	Lots of other data
Chris	Jones	Lots of other data
Diana	Brown	Lots of other data
Erica	York	Lots of other data
Joe	Anders	Lots of other data
Larry	White	Lots of other data
Samuel	Iger	Lots of other data

If ordered on Fname, then easy to search for these values by binary search:

```
SELECT * FROM EMPLOYEE
WHERE Fname = 'Larry';
```

How do we search for values of Lname ? Use an index.

4

(Dense) Index

Index File

LName	Row
Anders	5
Brown	3
Iger	7
Jones	2
Smith	1
White	6
York	4

Data File

FName	LName	…
Abraham	Smith	Lots of other data
Chris	Jones	Lots of other data
Diana	Brown	Lots of other data
Erica	York	Lots of other data
Joe	Anders	Lots of other data
Larry	White	Lots of other data
Samuel	Iger	Lots of other data

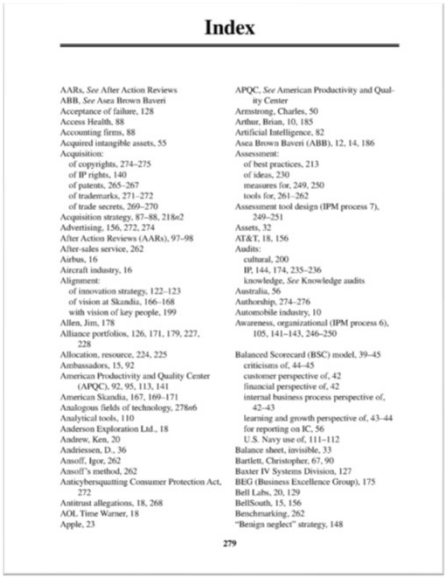
ordered on Lname to facilitate binary search of index

What happens if two people have the same Lname ?

5

Single-Level Index

- Index stores each value of the index field with list of pointers to all records with that field value
- Values in index are ordered
- Index can be searched (AKA “lookup”) using $O(\log n)$ binary or $O(\log\log n)$ interpolation search



Index

AARs, *See* After Action Reviews
ABB, *See* Asea Brown Baveri
Acceptance of failure, 128
Access Health, 88
Accounting firms, 88
Acquired intangible assets, 55
Acquisition:
  of copyrights, 274–275
  of IP rights, 140
  of patents, 265–267
  of trademarks, 271–272
  of trade secrets, 269–270
Acquisition strategy, 87–88, 218n2
Advertising, 156, 272, 274
After Action Reviews (AARs), 97–98
After-sales service, 262
Airbus, 16
Aircraft industry, 16
Alignment:
  of innovation strategy, 122–123
  of vision at Skandia, 166–168
  with vision of key people, 199
Allen, Jim, 178
Alliance portfolios, 126, 171, 179, 227, 228
Allocation, resource, 224, 225
Ambassadors, 15, 92
American Productivity and Quality Center (APQC), 92, 95, 113, 141
American Skandia, 167, 169–171
Analogous fields of technology, 278n6
Analytical tools, 110
Anderson Exploration Ltd., 18
Andrew, Ken, 20
Andriessen, D., 36
Ansoff, Igor, 262
Ansoff's method, 262
Anticybersquatting Consumer Protection Act, 272
Antitrust allegations, 18, 268
AOL Time Warner, 18
Apple, 23
APQC, *See* American Productivity and Quality Center
Armstrong, Charles, 50
Arthur, Brian, 10, 185
Artificial Intelligence, 82
Asea Brown Baveri (ABB), 12, 14, 186
Assessment:
  of best practices, 213
  of ideas, 230
  measures for, 249, 250
  tools for, 261–262
Assessment tool design (IPM process 7), 249–251
Assets, 32
AT&T, 18, 156
Audits:
  cultural, 200
  IP, 144, 174, 235–236
  knowledge, *See* Knowledge audits
Australia, 56
Authorship, 274–276
Automobile industry, 10
Awareness, organizational (IPM process 6), 105, 141–143, 246–250

Balanced Scorecard (BSC) model, 39–45
  criticisms of, 44–45
  customer perspective of, 42
  financial perspective of, 42
  internal business process perspective of, 42–43
  learning and growth perspective of, 43–44
  for reporting on IC, 56
  U.S. Navy use of, 111–112
Balance sheet, invisible, 33
Bartlett, Christopher, 67, 90
Baxter IV Systems Division, 127
BEG (Business Excellence Group), 175
Bell Labs, 20, 129
BellSouth, 15, 156
Benchmarking, 262
"Benign neglect" strategy, 148

279

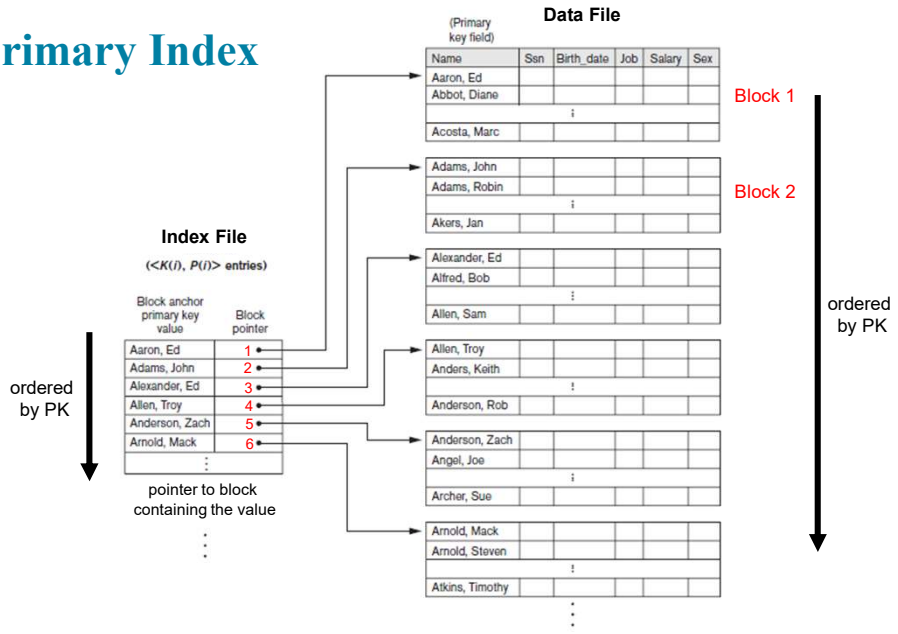
6

Indexes

- Any field can be indexed – LName
- Any combination of fields can be indexed - (LName, FName)
  - Any prefix of the index can be used for lookup
- Multiple indexes can be constructed for a single relation
- Most indexes are based on:
  - Tree data structures
  - Hash functions

7

(Sparse) Primary Index



8

Types of Single-Level Indexes

Primary index
- Specified on a key field
- Data file ordered on the field
- Typically is sparse (on blocks)
- Can have only single primary index

Secondary index
- Can be specified on any field
- Can have multiple secondary indexes

9

Example

Suppose that we have an ordered file with r = 1,000,000 records stored on a disk with block size B = 4K = 4,096 bytes. Any block can be accessed randomly and block access time dominates everything else.

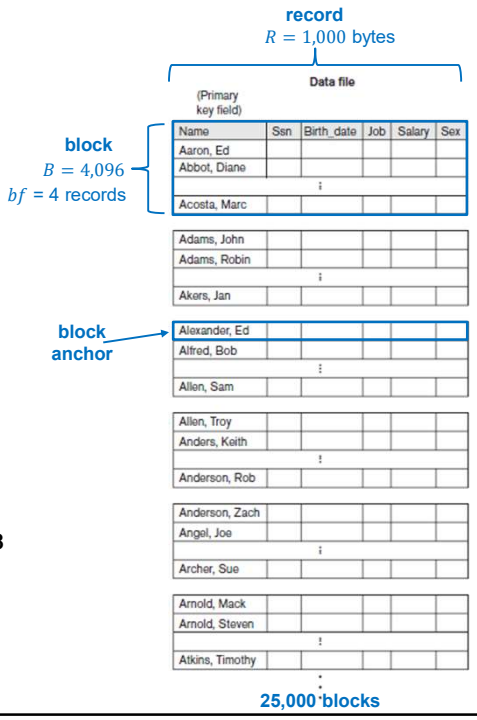
The records are of fixed size, with record length R = 1,000 bytes.

The blocking factor for the file would be

$$bf = B/R = \lfloor 4{,}096/1{,}000 \rfloor = \mathbf{4} \text{ records per block.}$$

The number of blocks needed for the file is $b = \frac{R}{bf} = \frac{1{,}000{,}000}{4} = \mathbf{250{,}000}$ blocks.

A binary search on the data file would need approximately $\log_2 b \approx \mathbf{18}$ block accesses.



10

Example

Now suppose that the ordering key field of the file consumes $V = 9$ bytes.

A block pointer consumes $P = 6$ bytes, and we have constructed a primary index for the file.

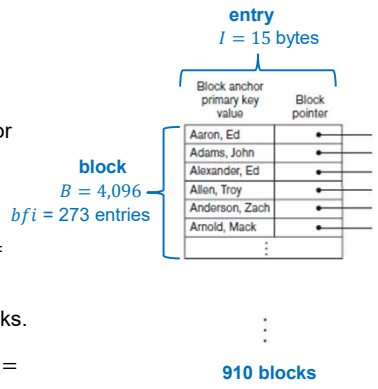
The size of each index entry is $I = 9 + 6 = 15$ bytes, so the blocking factor for the index is

$$bfi = B/I = 4{,}096/15 = 273 \text{ entries per block.}$$

The total number of index entries Ri is the number of blocks in the data file = $250,000$.

The number of index blocks is hence $bi = Ri/bfi = 250{,}000/273 = 910$ blocks.

To perform a binary search on the index file would require $\log_2 bi = \log_2 910 = 10$ block accesses.



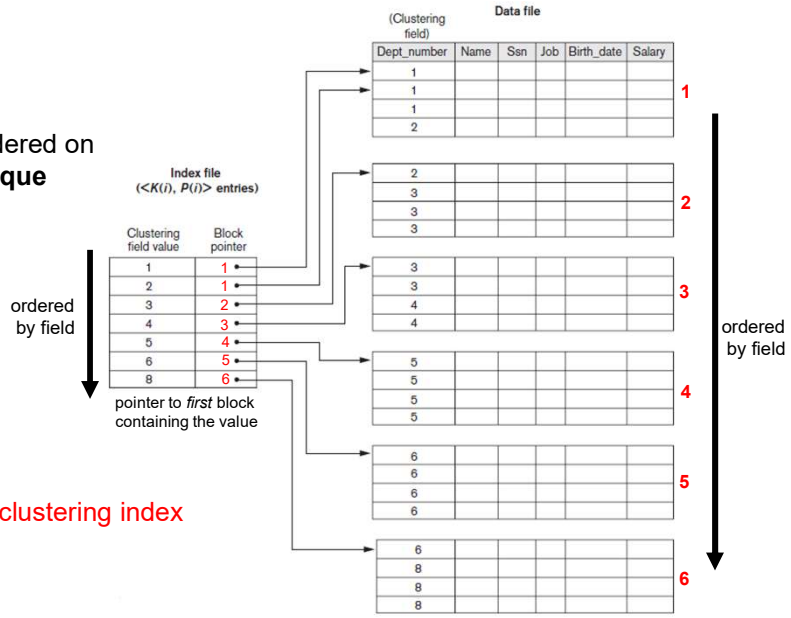
11

Clustering Index

File records are physically ordered on a **nonkey field without a unique value** for each record

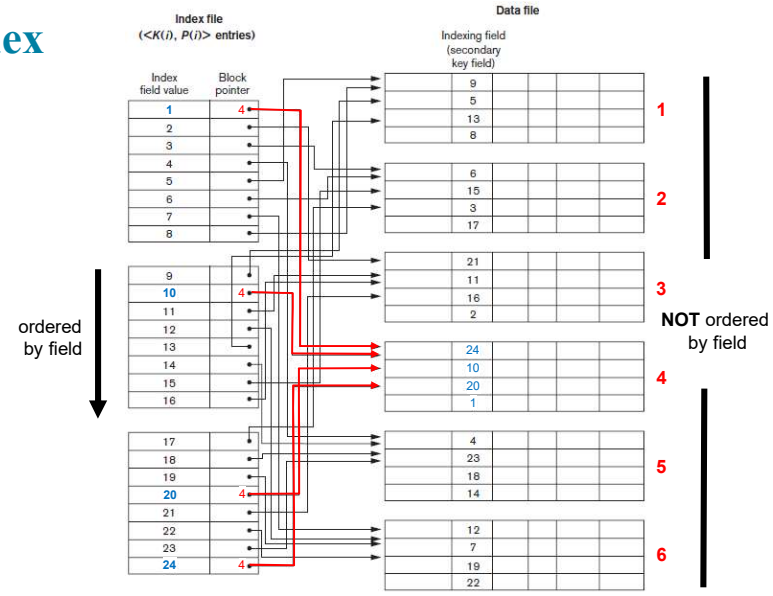
A clustering index on the Dept_number ordering nonkey field of **EMPLOYEE**

A relation can only have **one** clustering index (if there is no primary index)



12

Secondary Index (dense)



cell	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
key value	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
block	4	3	2	5	1	2	6	1	1	4	3	6	1	5	2	…

13

Single-Level Index

cell	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
key value	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
block	4	3	2	5	1	2	6	1	1	4	3	6	--	5	2	…
					3											

Easy to perform binary search on key value.

Lookup and *range query* = $O(\log n)$ time.

Main problem: insertion and deletion of records may require structural changes.

Example: Add(5.5,xx), Delete(13)

cell	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	…
key value	1	2	3	4	5	5.5	6	7	8	9	10	11	12	14	15	…
block	4	3	2	5	1	3	2	6	1	1	4	3	6	5	2	…

Solutions:

- Use unordered overflow file
- Use linked list of overflow records
- Periodic re-indexing

14

Indexes in SQL

```
CREATE [UNIQUE|CLUSTER] INDEX <index_name>
ON <table_name> (<column_name> [<order>]
                {,<column name> [<order>]})
```

create index

- A relation can have only *one* clustered index (because of the ordering)
- `<order>` can be `ASC` or `DESC`
- Index can be on single column or multiple columns. If multiple columns – this is useful when any prefix of the columns is searched for
- SQLite automatically creates primary indexes on key and unique attributes

```
PRAGMA index_list('<table_name>')
PRAGMA index_info('<index_name>')
```

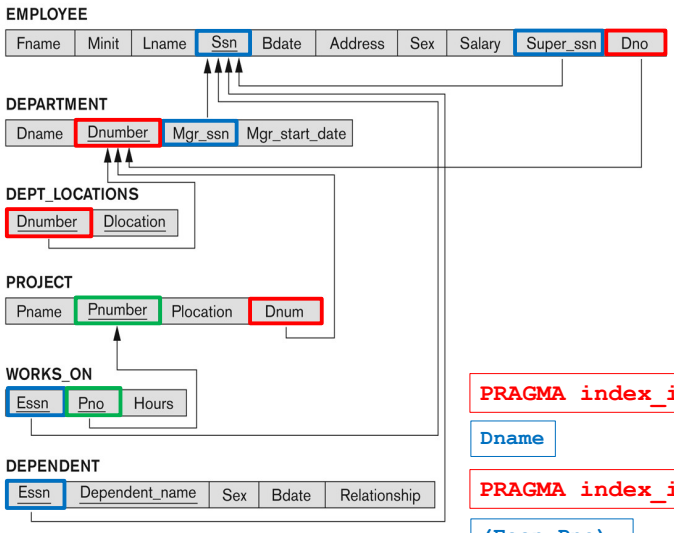
info on indexes

```
DROP <index_name>
```

remove index

15

COMPANY




```
PRAGMA index_list('DEPARTMENT')
sqlite_autoindex_DEPARTMENT_2 u
sqlite_autoindex_DEPARTMENT_1 pk
PRAGMA index_list('PROJECT')
sqlite_autoindex_PROJECT_2 u
sqlite_autoindex_PROJECT_1 pk
PRAGMA index_list('WORKS_ON')
sqlite_autoindex_WORKS_ON_1 pk
PRAGMA index_info('sqlite_autoindex_DEPARTMENT_2')
Dname
PRAGMA index_info('sqlite_autoindex_WORKS_ON_1')
(Essn,Pno)
```

16

Indexes are Fast Lookup

```
EXPLAIN QUERY PLAN
SELECT * FROM EMPLOYEE
WHERE Ssn = '987987987';
```

not efficient - $O(n)$

```
for E in EMPLOYEE
   if E.Ssn == '987987987'
      print(e.*);
```

efficient by index - $O(\log n)$

```
E = search('987987987',[EMPLOYEE.Ssn]);
if !empty(E) print(E.*);
```

```
SEARCH EMPLOYEE USING INDEX sqlite_autoindex_EMPLOYEE_1 (Ssn=?)
```

17

Indexes Speed Up Joins

```
EXPLAIN QUERY PLAN
SELECT E.Lname, D.Dname
FROM EMPLOYEE E, DEPARTMENT D
WHERE E.Dno = D.Dnumber;
```

not efficient - $O(nm)$

```
for E in EMPLOYEE
   for D in DEPARTMENT
      if E.Dno = D.Dnumber
         print(E.Lname, D.Dname);
```

efficient by index - $O(n \log m)$

```
for E in EMPLOYEE
   D = search(E.Dno,
         [DEPARTMENT.Dnumber]);
   if !empty(D)
      print(E.Lname,[D.Dname]);
```

not efficient - $O(nm)$

```
for D in DEPARTMENT
   E = search(D.Dnumber,
            [EMPLOYER.Dno]);
   if !empty(E)
      print(E.Lname,[D.Dname]);
```

```
SCAN E
SEARCH D USING INDEX sqlite_autoindex_DEPARTMENT_1 (Dnumber=?)
```

18

Query: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
EXPLAIN QUERY PLAN
SELECT P.Pnumber, P.Dnum, E.Lname, E.Address, E.Bdate
FROM PROJECT P, EMPLOYEE E, DEPARTMENT D
WHERE  P.Plocation ='Stafford' AND
       P.Dnum = D.Dnumber      AND
       D.Mgr_ssn = E.Ssn;
```

```
SCAN P
SEARCH D USING INDEX sqlite_autoindex_DEPARTMENT_1 (Dnumber=?)
SEARCH E USING INDEX sqlite_autoindex_EMPLOYEE_1 (Ssn=?)
```

19

Query: Create a list of employees working on projects, with their hours.

```
EXPLAIN QUERY PLAN
SELECT E.Fname, E.Lname, P.Pname, W.Hours
FROM   EMPLOYEE E, PROJECT P, WORKS_ON W
WHERE  E.Ssn = W.Essn   AND
       W.Pno = P.Pnumber;
```

Prefix of PK index

```
SCAN E
SEARCH W USING INDEX sqlite_autoindex_WORKS_ON_1 (Essn=?)
SEARCH P USING INDEX sqlite_autoindex_PROJECT_1 (Pnumber=?)
```

20

Query: Show the resulting salaries if every employee working on the 'ProductX' project is given a 10% raise.

```
EXPLAIN QUERY PLAN
SELECT E.FName, E.LName, 1.1*E.Salary AS NewSalary
FROM   EMPLOYEE E, WORKS_ON W, PROJECT P
WHERE  P.Pname = 'ProductX' AND
       W.Pno = P.Pnumber    AND
       E.Ssn = W.Essn;
```

```
SEARCH P USING INDEX sqlite_autoindex_PROJECT_2 (Pname=?)
SCAN E
SEARCH W USING COVERING INDEX sqlite_autoindex_WORKS_ON_1
        (Essn=? AND Pno=?)
```

Entire PK is used both in SELECT and WHERE

21

Temporary Indexes

Sometimes it pays to create an index for a single query.
Cost = $O(n \log n)$.

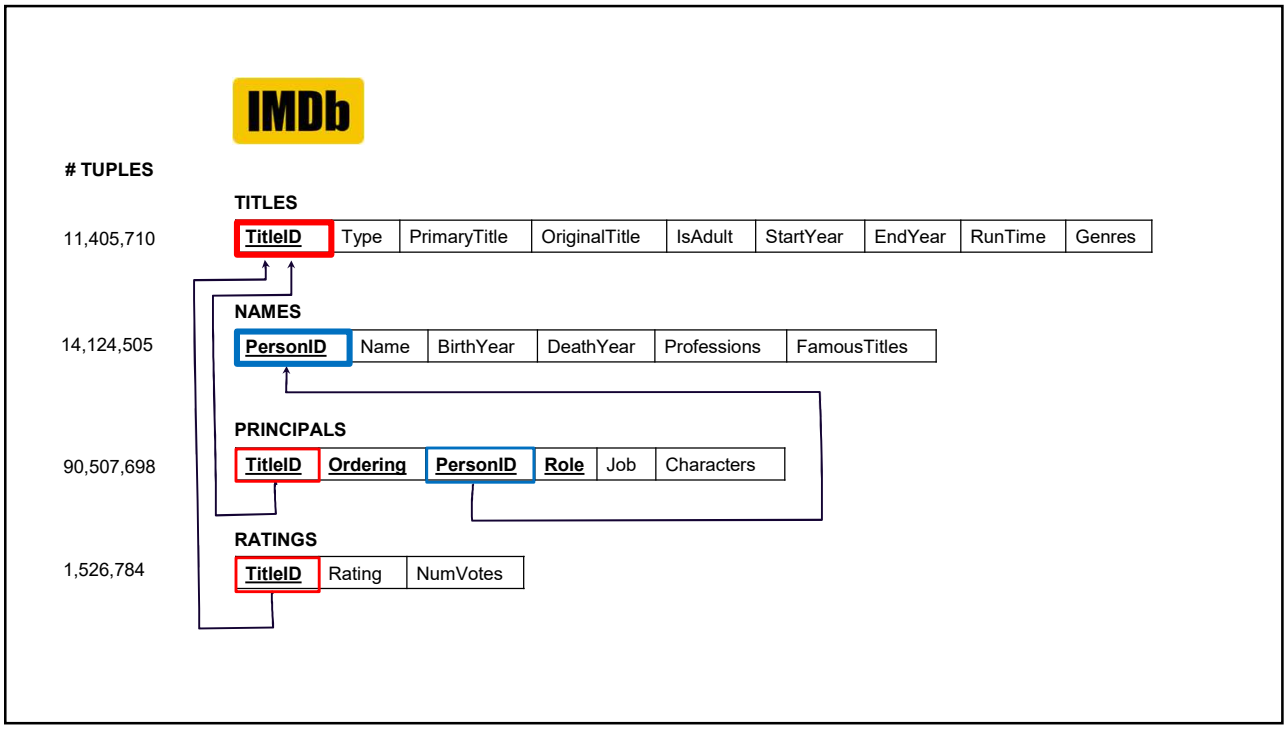
Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Alicia	Zalaya	25000
Jennifer	Wallace	43000
Ramesh	Narayan	38000
Joyce	English	25000
Ahmad	Jabbar	25000
James	Borg	55000

```
EXPLAIN QUERY PLAN
SELECT E1.FNAME, E1.LNAME, E1.SALARY
FROM   EMPLOYEE E1,EMPLOYEE E2
WHERE  E1.LNAME = E2.LNAME;
```

```
SCAN E1
SEARCH E2 USING AUTOMATIC COVERING INDEX (Lname=?)
```

Cost = $O(n \log n)$ instead of $O(n^2)$.

22



23

SQLite automatically creates an index for every unique or key attribute(s).

Query: List the details of the movies “The Good, the Bad and the Ugly” and “Million Dollar Baby”.

Using an indexed (key) attribute:

```
SELECT * FROM TITLES
WHERE  TitleID = 'tt0060196' OR
       TitleID = 'tt0405159';
```

n = size(TITLES)

```
SEARCH TITLES USING INDEX
       sqlite_autoindex_TITLES_1 (TitleID=?)
```

$O(\log n)$

Using a non-indexed attribute:

```
SELECT * FROM TITLES
WHERE PrimaryTitle = 'The Good, the Bad and the Ugly' OR
      PrimaryTitle = 'Million Dollar Baby'
```

```
SCAN TITLES
```

$O(n)$

24

Query: List the details of the movies from 1966 in which there was a character called “Blondie”

This requires a search on two non-indexed attributes:

```
SELECT * FROM TITLES T, PRINCIPALS P
WHERE P.TitleID = T.TitleID          AND
      P.Characters = '["Blondie"]'   AND
      T.StartYear = 1966;
```

n = size(TITLES)
m = size(PRINCIPALS)

```
for P in PRINCIPALS
   if P.Characters = '["Blondie"]'
      T = search(P.TitleId,TITLES.TitleId);
       if !empty(T) && T.StartYear = 1966
             print(T.*,P.*);
```

```
SCAN P
SEARCH T USING INDEX
   sqlite_autoindex_TITLES_1 (TitleID=?)
```

$O(m + k_P \log n)$ k_P= # qualifying P’s

```
for T in TITLES
   if T.StartYear = 1966
      P = search(T.TitleId,PRINCIPALS.TitleId);
       if !empty(P) && P.Characters = '["Blondie"]')
             print(T.*,P.*);
```

possible because TitleID is prefix of index

```
SCAN T
SEARCH P USING INDEX
   sqlite_autoindex_PRINCIPALS_1 (TitleID=?)
```

$O(n + k_T \log m)$ k_T= # qualifying T’s

25

Query: List the details of the movies from 1966 in which there was a character called “Blondie”

```
SELECT * FROM TITLES T, PRINCIPALS P
WHERE P.TitleID = T.TitleID          AND
      P.Characters = '["Blondie"]'   AND
      T.StartYear = 1966;
```

n = size(TITLES)
m = size(PRINCIPALS)

After an index is created on each of these two attributes:

```
CREATE INDEX CHAR  ON PRINCIPALS(Characters);
```

```
P = search('["Blondie"]',[PRINCIPALS.Characters]);
if !empty(P)
   T = search(P[i].TitleID,[TITLES.TitleID]);
   if !empty(T) && T.StartYear = 1966
       print(T.*,P.*);
```

```
SEARCH P USING INDEX CHAR (Characters=?)
SEARCH T USING INDEX
   sqlite_autoindex_TITLES_1 (TitleID=?)
```

$O(\log m + k_P \log n)$ k_P= # qualifying P’s

```
CREATE INDEX SYEAR ON TITLES(StartYear);
```

```
T = search(1966,[TITLES.StartYear]);
if !empty(T)
   P = search(T[i].TitleID,[PRINCIPALS.TitleID]);
   if !empty(P) && P.Characters = '["Blondie"]'
            print(T.*,P.*);
```

```
SEARCH T USING INDEX SYEAR (StartYear=?)
SEARCH P USING INDEX
   sqlite_autoindex_PRINCIPALS_1 (TitleID=?)
```

$O(\log n + k_T \log m)$ k_T= # qualifying T’s

26

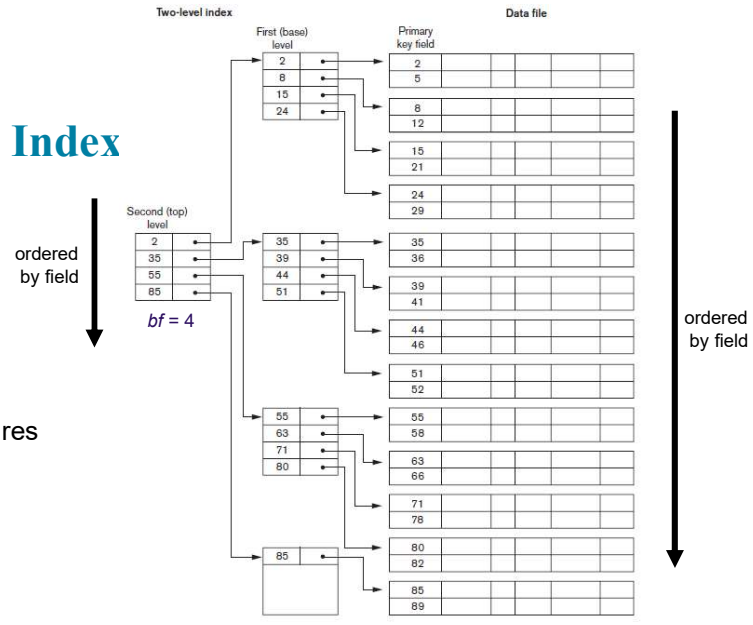
Main Objective in Indexing a Table:

Given the value k of an attribute, identify very quickly the rows containing k in the table, or indicate that none exist.

The index should be easy to maintain if the table is dynamic.

27

(Sparse) Multi-Level Primary Index



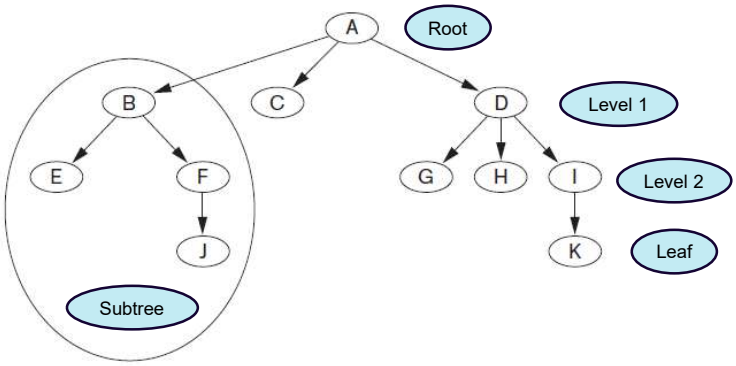
Searching a multilevel index requires $\log_{bf} bi$ block accesses

Insert and delete is easier.

28

Tree Data Structure

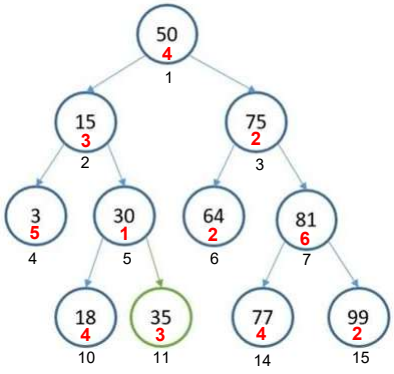
An unbalanced tree



29

Storing a Binary (Search) Tree

cell	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
key value	50	15	75	3	30	64	81			18	35			77	99
block	4	3	2	5	1	2	6			4	3			4	2



Children of node in cell i are in cells $2i$ and $2i+1$

Values in left/right subtree are smaller/larger

Search, Add and Delete require $O(\log n)$ time

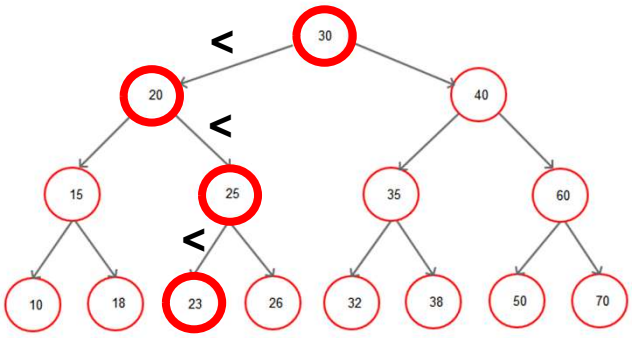
Construction requires $O(n \log n)$ time.

Some effort is required to keep tree balanced

30

Binary Search Tree (BST)

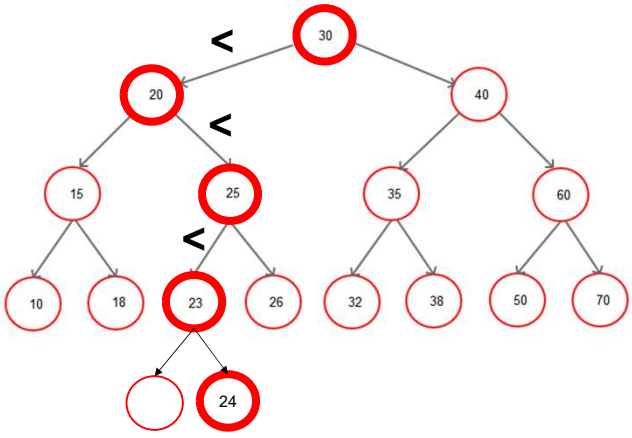
Search for 24



31

Update of Binary Search Tree

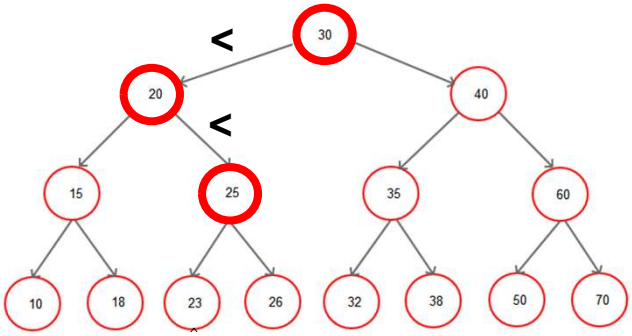
Add 24



32

Update of Binary Search Tree

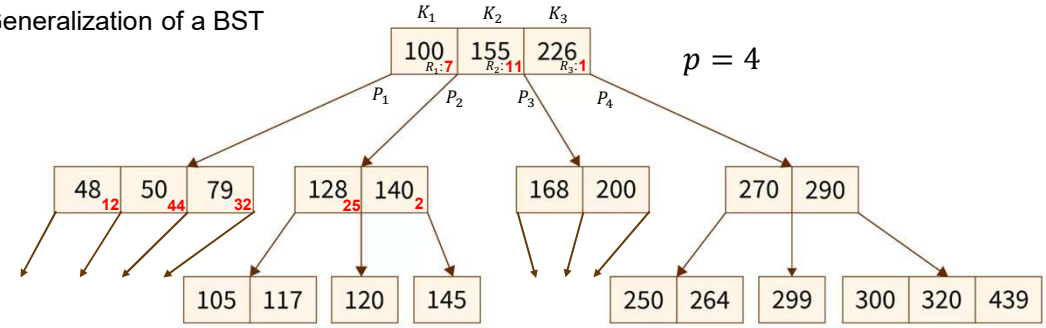
Delete 25



33

B-Tree

Generalization of a BST



1. Every node has at most p children.
2. **Every node, except for the root and the leaves, has at least $\lceil p/2 \rceil$ children.**
3. The root node has at least two children unless it is a leaf.
4. **All leaves are at the same level.**
5. A non-leaf node with k children contains $k-1$ keys.

1. Within each node, $K_1 < K_2 < \cdots < K_{p-1}$.
2. For all values X in the subtree pointed at by P_i,

$$\begin{array}{ll} X < K_i & \text{for } i = 1 \\ K_{i-1} < X < K_i & \text{for } 1 < i < p \\ K_{i-1} < X & \text{for } i = p \end{array}$$

Each key has a pointer to a data block/row

34

B-Tree

- **Balanced** p-way tree
- Generalization of BST in which a node can have more than one key and more than two children
- Maintains sorted keys
- **Takes advantage of storage block structure.**
- **Space wasted by deletion never becomes excessive, because each node is at least half-full**
- Update requiring significant change to tree is rare

35

B-Tree Complexity

Best that p is **odd.** p is typically hundreds.

Assuming a p-way B-Tree T with n keys:

min height$(T) = \lceil \log_p(n+1) \rceil - 1$

max height$(T) = \left\lfloor \log_t \left(\frac{n+1}{2} \right) \right\rfloor, \; t = \left\lceil \frac{p}{2} \right\rceil$

$n = 100{,}000, p = 7$

$\min(h) = 5$
$\max(h) = 6$

Search, Add, Delete on B-Trees cost $O(\log_p n)$ runtime.

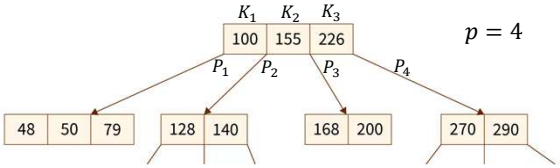
Construction of a B-Tree costs $O(n \log_p n)$ runtime.

36

B-Tree Search

Similar to search in binary tree:

```
search(T, k)
{
    find smallest i such that T.K[i] ≥ k
    if T.K[i] == k
        return T.R[i]
    else
        if isleaf(T)
            return NULL
        else
            return search(T.P[i], k)
}
```

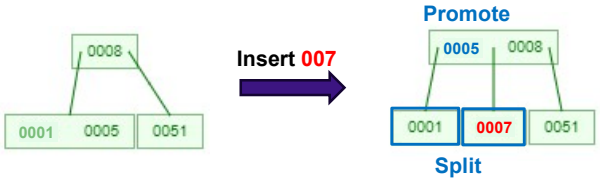


37

B-Tree Insert

- An insertion into a node that **is not full** is efficient
- If a node is **full** the insertion causes an **overflow** and triggers a split-promotion:
  - **Split** the overflow node into two half-full nodes
  - **Promote** the middle key to a parent node

$p = 3$

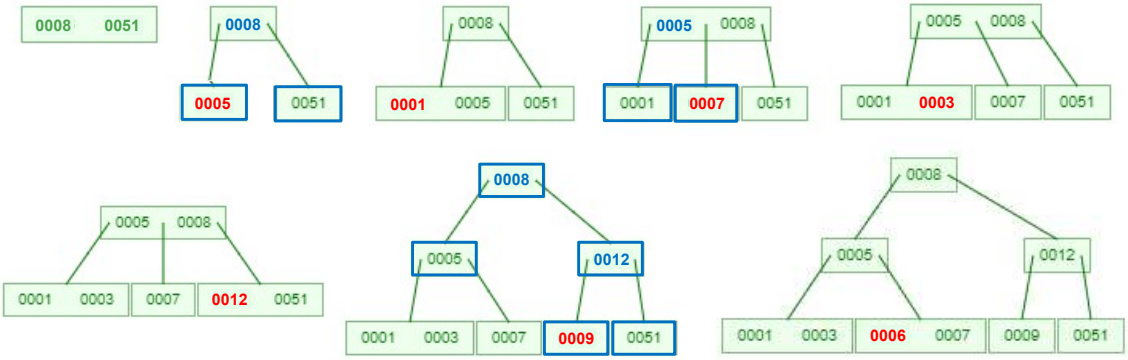


If the promotion causes additional overflow, then repeat the split-promotion

38

Example

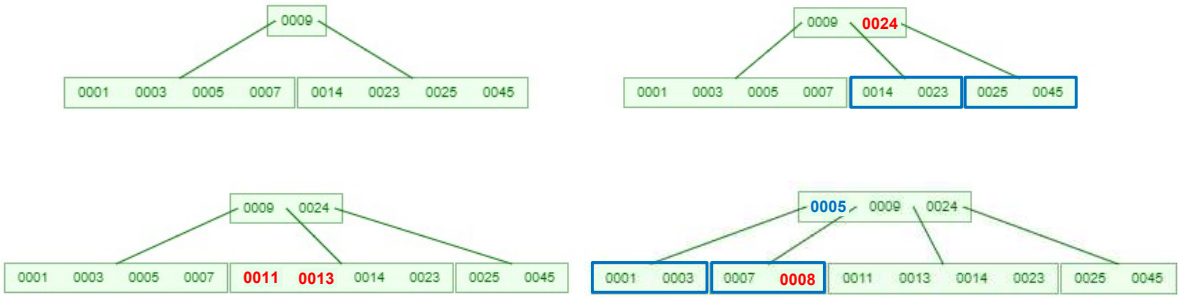
Insert the values **5, 1, 7, 3, 12, 9, 6** into a B-Tree of order $p = 3$. (Each node can have no more than two values).



39

Example

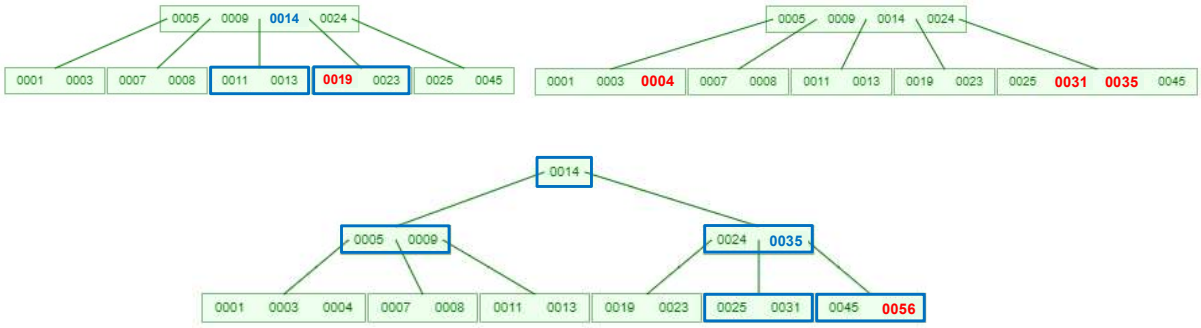
Insert the values **24, 13, 11, 8** into a B-Tree of order $p = 5$.



40

Example

Continue to insert the values **19, 4, 31, 35, 56**



41

B-Tree Delete

- A deletion into a node that does not become less than half full is efficient
- If a deletion causes a node to become less than half full, there is an **underflow** and the node must be **merged** with neighboring nodes:

  CASE I: The key is in a leaf node

  CASE II: The key is in an internal node

  CASE III: The key is in the root node

42

B-Tree Delete

CASE III: If the key is in the root node

Replace with the maximum element of the in-order predecessor subtree

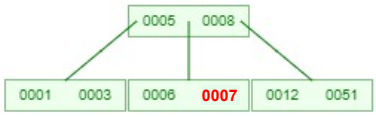
If, after deletion, the target has less than min keys, then the target node will borrow max value from its sibling via the sibling's parent.

The max value of the parent will be taken by a target, but with the nodes of the max value of the sibling.

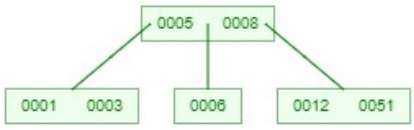
43

Example

$p = 3$

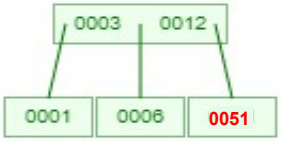


Delete 7

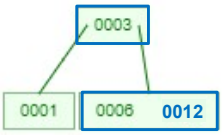


44

Example

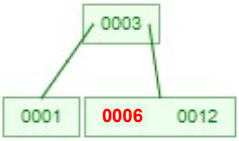


Delete 51

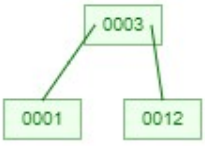


45

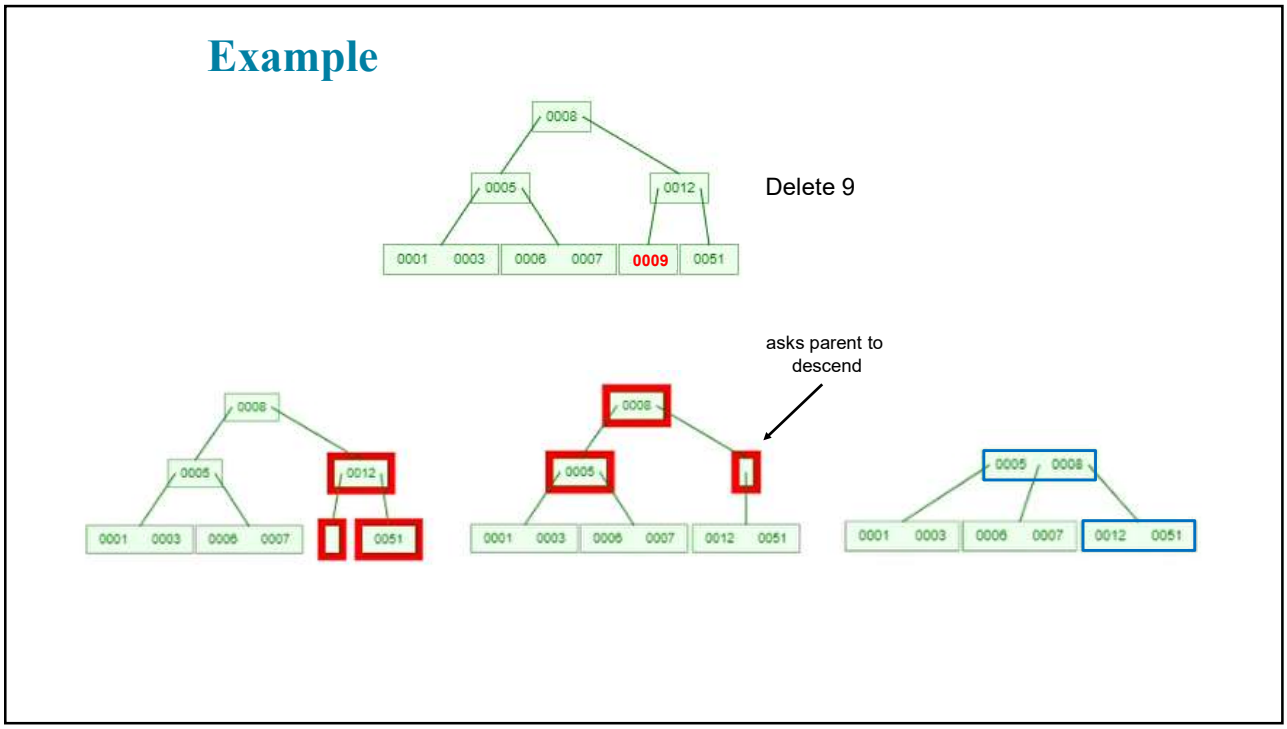
Example



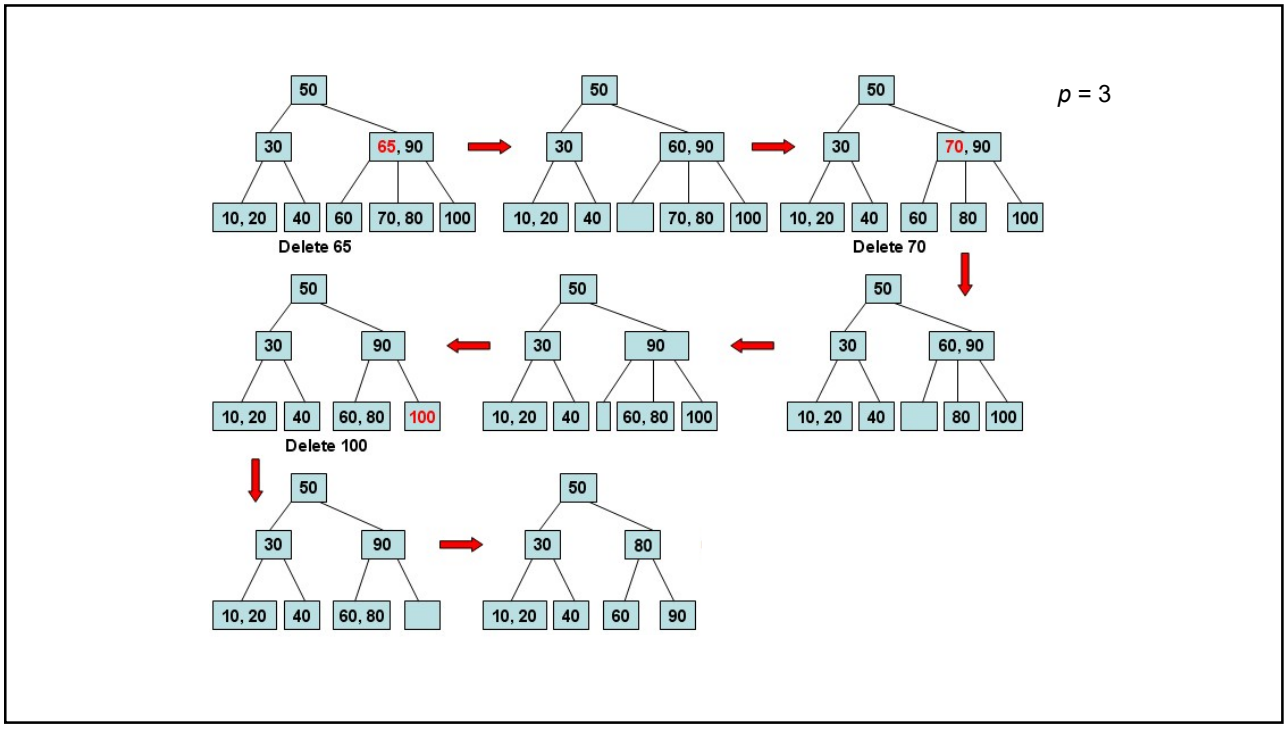
Delete 6



46


Example
0008
0005
0012
0001 0003 0006 0007 0009 0051
Delete 9
asks parent to descend
0008
0005
0012
0001 0003 0006 0007
0051
0008
0005
0001 0003 0006 0007 0012 0051
0005 0008
0001 0003 0006 0007 0012 0051

47


p = 3
50
30
65, 90
10, 20 40 60 70, 80 100
Delete 65
50
30
60, 90
10, 20 40 70, 80 100
50
30
70, 90
10, 20 40 60 80 100
Delete 70
50
30
60, 90
10, 20 40 80 100
50
30
90
10, 20 40 60, 80 100
50
30
90
10, 20 40 60, 80 100
Delete 100
50
30
90
10, 20 40 60, 80
50
30
80
10, 20 40 60 90

48

Hashing

49

Hash Functions

A *hash function* of the key k of a relation of n rows gives the id of the bucket containing k in the *hash table* having N buckets

row	name	data
1	Bob	…
2	Charles	…
3	Elina	…
4	Alice	…
5	Elisa	…
6	Greg	…
7	Lance	…
8	Nancy	…

$$h_1(k) = [\textbf{first letter of } k] \textbf{ mod } N$$

0	1	2	3	4	5	6
Greg Nancy	Alice	Bob	Charles		Elina Elisa Lance	

$N = 7$

$$h_2(k) = [\textbf{second letter of } k] \textbf{ mod } N$$

0	1	2	3	4	5	6
	Lance Nancy Charles Bob			Greg	Elina Alice Elisa	

Should be easy to compute h - $O(1)$
On the average, there will be n/N keys per bucket

50

Hash-based Index

Hash-based indexes are best for equality selections
(= search/lookup)

row	name	data
1	Bob	…
2	Charles	…
3	Elina	…
4	Alice	…
5	Elisa	…
6	Greg	…
7	Lance	…
8	Nancy	…

$N = 7$

Search(HT, k)
$i = h(k)$;
for j = 1 to len$(HT(i).\text{key})$
if $HT(i).\text{key}(j) == k$
return $HT(i).\text{row}(j)$
return *NULL*;

0		1		2		3		4		5		6	
key	row	key	row	key	row	key	row	key	row	key	row	key	row
Greg Nancy	6 8	Alice	4	Bob	1	Charles	2			Elina Elisa Lance	3 5 7		

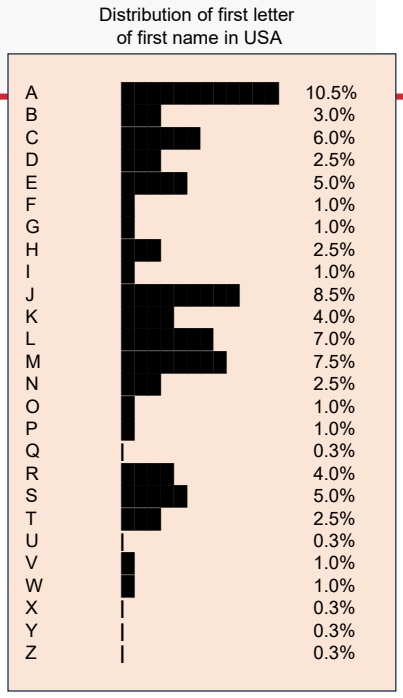
Cannot support range searches

collision (bucket overflow)!

51

Overflows

- Bucket overflow occurs if
  - Insufficient buckets
  - Skew in the distribution of records
    - Multiple records have the same key value
    - The hash function produces non-uniform distribution of key values
- Although the chances of bucket overflow can be reduced, it cannot be eliminated entirely
- Collision resolution is required



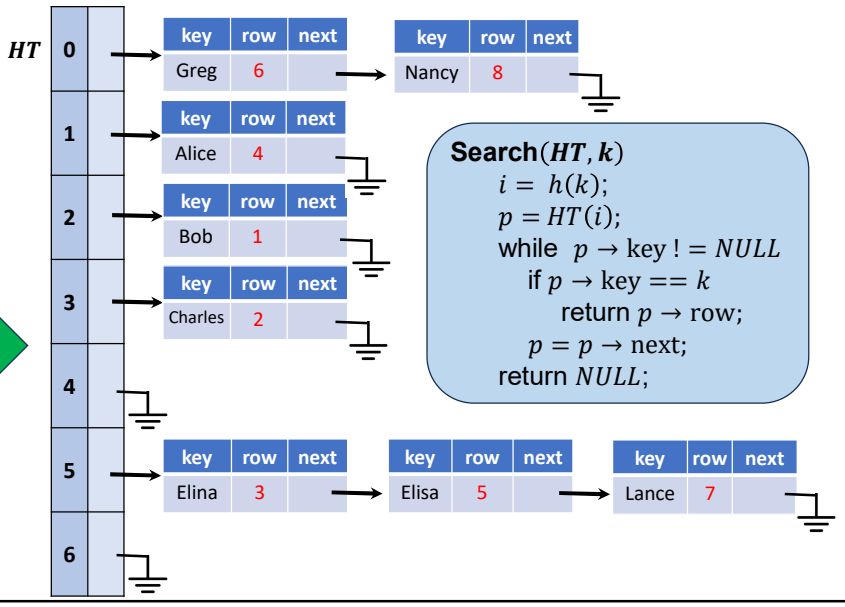
52

Static Hashing: Chaining Buckets

- Number of primary buckets is fixed
- Use overflow buckets as needed to deal with collisions
- Easy to maintain

row	name	data
1	Bob	…
2	Charles	…
3	Elina	…
4	Alice	…
5	Elisa	…
6	Greg	…
7	Lance	…
8	Nancy	…

$N = 7$



Search(HT, k)
$i = h(k)$;
$p = HT(i)$;
while $p \rightarrow \text{key} != NULL$
if $p \rightarrow \text{key} == k$
return $p \rightarrow \text{row}$;
$p = p \rightarrow \text{next}$;
return $NULL$;

Static Hashing: Open Addressing

- Number of primary buckets is fixed
- In case of collision – use the next available bucket
- Use *DELETED* to mark a bucket after removal

row	name	fl
1	Bob	2
2	Charles	3
3	Elina	5
4	Alice	1
5	Elisa	5
6	Greg	7
7	Lance	12
8	Nancy	14

Search(HT, k)
$j = h(k)$;
while $HT(j).\text{K} != NULL$
if $HT(j).\text{K} == k$
return $HT(j).\text{R}$;
$j = j + 1$;
return $NULL$;

$N = 12$

HT

0		1		2		3		4		5		6		7		8		9		10		11	
K	R	K	R	K	R	K	R	K	R	K	R	K	R	K	R	K	R	K	R	K	R	K	R
Lance	7	Alice	4	Bob	1	Charles	2	Nancy	8	Elina	3	Elisa	5	Greg	6								

Static Hashing: Comparison

Feature	Chaining	Open Addressing
Best when …	Num of keys unknown	Num of keys bounded
Load Factor (α)	Can be > 1.0	Should stay < 0.7
Search time	O(1+chain length)	$O(1/(1-\alpha))$
Memory	Uses more per key	Uses less per key

55